Escuela de Computación
Ingeniería en Computación
IC-4700 - Lenguajes de Programación

Proyecto 1: Scheme - Erlang

Programación Genética en Scheme con
Paralelización en Erlang

Dayana Rojas Campos
2024101499

Nicole Bou Espinoza
2024178699

Alajuela, Costa Rica
18 de abril del 2026.

Índice

1. Resumen **2**

2. Introducción **2**

3. Marco Teórico **2**

4. Descripción de la Solución **2**

5. Resultados **3**

6. Conclusiones **3**

7. Bibliografía **4**

1. Resumen

2. Introducción

Con el propósito de poder explorar mejor el paradigma funcional, el presente proyecto combina a dos de los lenguajes principales de esta área: Erlang y Scheme. Esta mezcla se realiza con el objetivo de poder profundizar en la programación genética en conjunto con la computación paralela. La idea principal de este proyecto es utilizar Scheme para poder generar árboles sintácticos y, a partir de ahí, irlos cruzando y continuando el ciclo evolutivo para encontrar funciones matemáticas específicas a partir de una lista de puntos. Como la parte del fitness puede ser más lenta, ahí es donde entra Erlang como una clase de motor de paralelización. Esto va a permitir dividir el trabajo pesado entre varios nodos que funcionan al mismo tiempo para aprovechar tanto como sea posible los recursos. En las secciones a continuación, se estará hablando más sobre la implementación y los resultados con comparativas entre nodos y tamaños de poblaciones.

3. Marco Teórico

Por la naturaleza de este proyecto, hay varios conceptos clave para tomar en cuenta. Primero que todo, se podría pensar en la programación genética. Esta es una técnica que permite generar programas de forma automática, en este caso, mediante representarlos como árboles sintácticos. Para esto, Scheme es ideal puesto que permite manipular dichos árboles antes de evaluarlos. Por otro lado, para que sea más veloz, se utilizan servidores y nodos con ayuda de Erlang. La idea de la paralelización es que un nodo central reparta las tareas entre los demás (en este caso, repartiría los individuos). Así cada nodo se encarga de una parte y contribuye a aligerar la carga. De esta forma, se reduce significativamente el tiempo de ejecución.

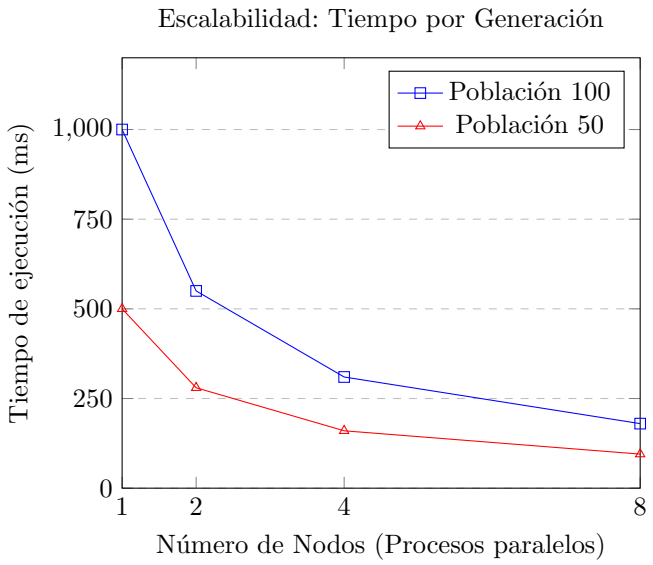
4. Descripción de la Solución

Como se mencionó con anterioridad, para este proyecto se utilizó Scheme para la parte del motor genético y Erlang para la parte de la distribución. Debido a la naturaleza del problema, ambos lenguajes debían poder comunicarse para cumplir con el siguiente recorrido: Scheme recibe los puntos y genera la población inicial; posteriormente, le pasa ambas cosas a Erlang. Este distribuye los datos en sus nodos y los manda a evaluar con Scheme; después de eso, Scheme le devuelve los resultados tras reemplazar x e y, y comparar con z. Con esto, Erlang evalúa el *fitness* y lo devuelve junto a la población para que se pueda seguir el ciclo evolutivo en Scheme. A partir de allí, ya se puede realizar el torneo para cruzar a ciertos individuos, con probabilidades de mutación.

Para la creación de los individuos, se hace un uso extensivo de las funciones de aleatoriedad de Scheme. Esta se utiliza para escoger la operación y las hojas, además de definir en qué momentos se genera un subárbol adicional en lugar de terminar la rama. También se utiliza en procesos como el torneo y el cruce; de esta manera, se escoge cuáles serán los padres de un nuevo individuo, de quién heredará la raíz y de qué padre heredará el hijo izquierdo (y, por descarte, el derecho). Incluso, al estar eligiendo los hijos, existe la oportunidad de que ocurra una mutación, donde se genera un nuevo subárbol aleatorio en lugar de ser uno heredado por alguno de sus padres.

La otra gran parte de este proyecto consistía en establecer la conexión entre los dos lenguajes. Para esto, se creó un puente basado en el paso de mensajes. Como ambos lenguajes manejan formatos diferentes, se debió encontrar una forma de transmitir la información en un formato que permitiera la recepción sin provocar errores de sintaxis. De esta forma, Scheme le envía a Erlang los puntos y los árboles para que este pueda distribuir el trabajo equitativamente entre los nodos para realizar cálculos en paralelo. Finalmente, Erlang recolecta todos los valores de *fitness* calculados y los envía de vuelta junto con el árbol correspondiente, permitiendo así que continúe el ciclo evolutivo.

5. Resultados



6. Conclusiones

- La implementación de un algoritmo de programación genética sobre un paradigma funcional como el de Scheme, facilita significativamente el manejo de árboles para la creación y la evaluación. La aleatoriedad es un

gran apoyo en cuanto a esto ya que nos permite tener variedad dentro de una misma población sin complicarse de forma excesiva el código, aprovechando la naturaleza del lenguaje.

- El uso de Erlang como herramienta de distribución demuestra que es posible dividir las tareas a otros nodos. La comunicación entre lenguajes diferentes presenta retos técnicos, la capacidad de procesar múltiples individuos de forma paralela es una ventaja clara para escalar el proyecto a poblaciones más grandes.

- El diseño de una interfaz basada en mensajes entre Scheme y Erlang permitió comprender la importancia de la serialización de datos. Lograr que un árbol generado en un lenguaje sea interpretado correctamente por otro es un ejercicio fundamental de interoperabilidad, resaltando que la arquitectura del sistema es tan importante como la lógica del algoritmo genético en sí.

7. Bibliografía

Racket Software Foundation, *Racket Documentation: The Racket Reference*, 2026. [En línea]. Disponible en: https://docs.racket-lang.org/reference/

Ericsson AB, *Erlang/OTP Documentation: Efficiency Guide and Parallel Programming*, 2026. [En línea]. Disponible en: https://www.erlang.org/doc/